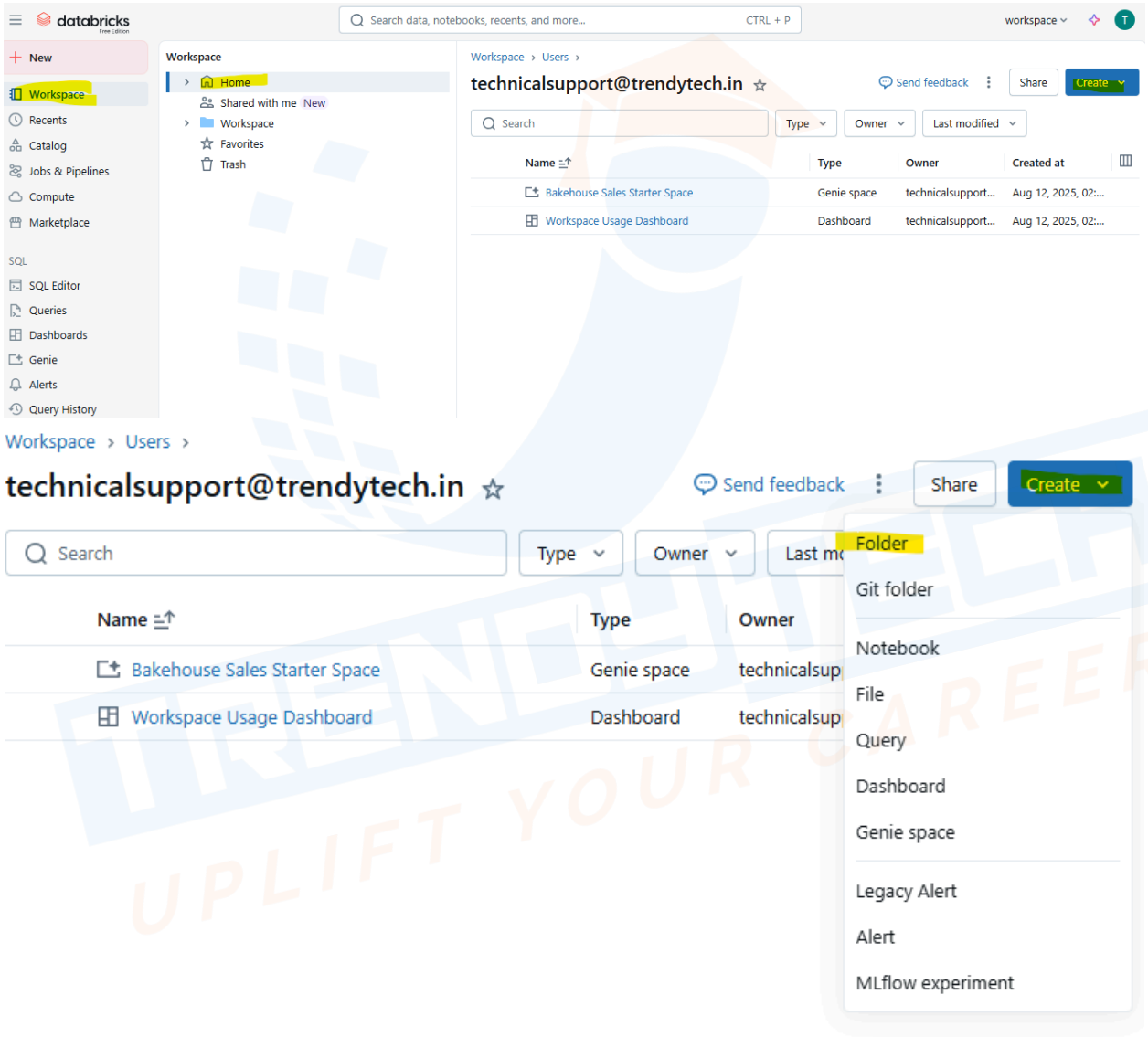
Databricks Free edition

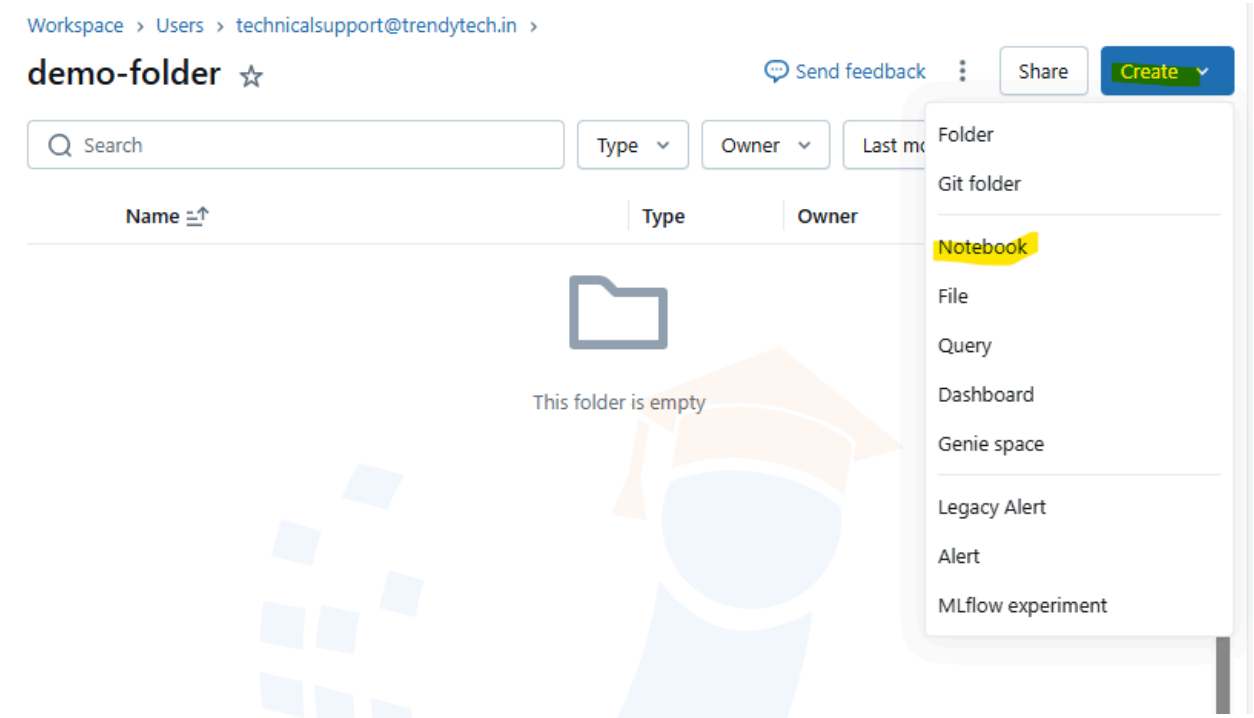
Login to databricks free edition using the below given link:
https://login.databricks.com/?tuuid=5d7b8c2e-9025-400f-a4dc-4503a4f3a349&dbx_source=direct&rl_aid=eb104bb6-df53-478c-8490-19841b044eec

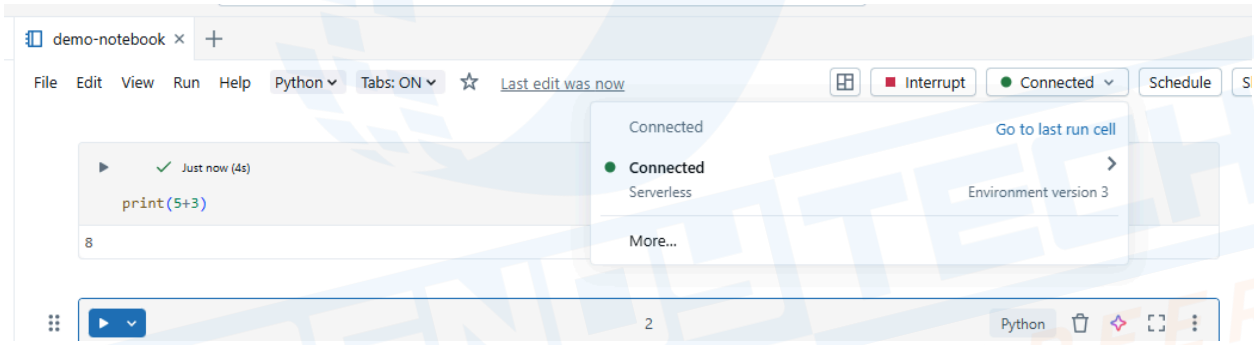
Session 4: Hands-on with databricks Notebooks
Steps to run a code in notebook:

Step: 1 Workspace → Home → Create → Folder



Once the folder is created,
Step 2: demo-folder → create → notebook



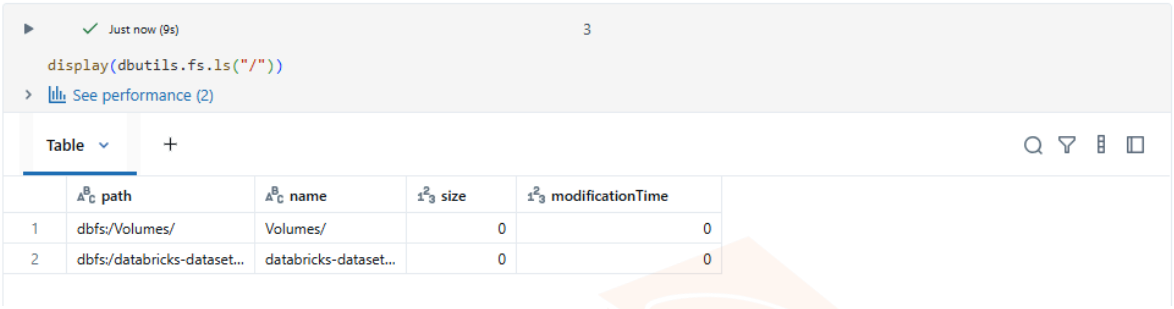


Step 3: Create Compute:
Only serverless computer is present in free edition and classic compute is not present

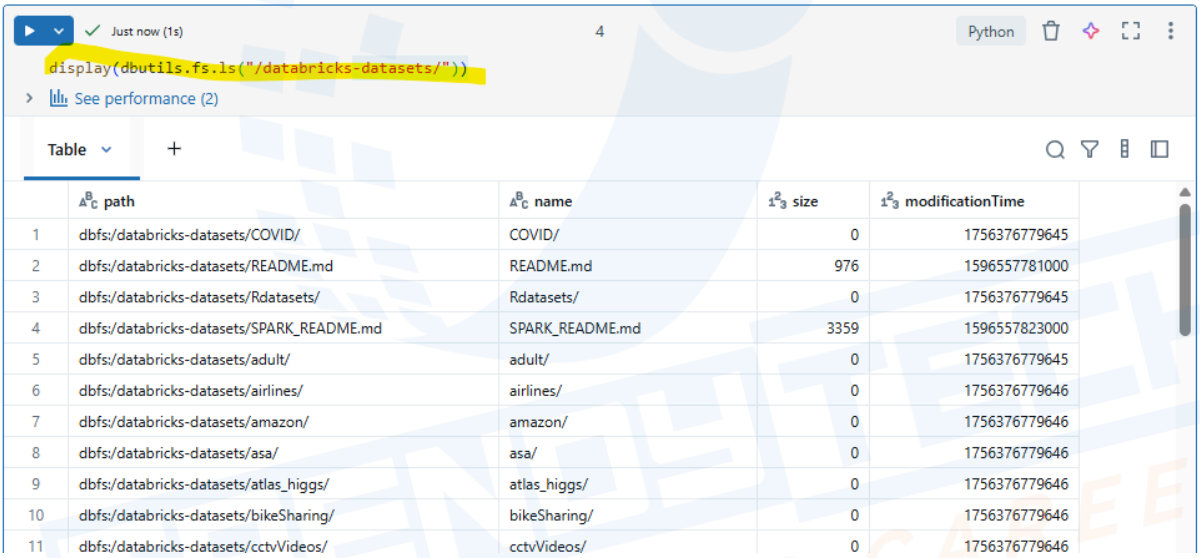
Here's how it works in free edition:

- You get one small cluster automatically provisioned for you.
- You can attach/detach notebooks to this cluster.
- You cannot change its size, runtime, autoscaling, or configuration.
- It times out after some inactivity and you may need to restart it.

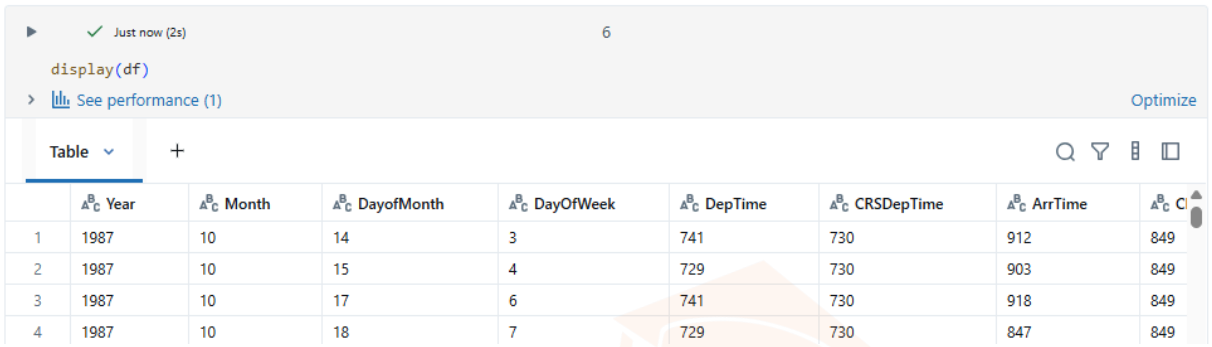
```
dbutils.fs.ls("/")
```



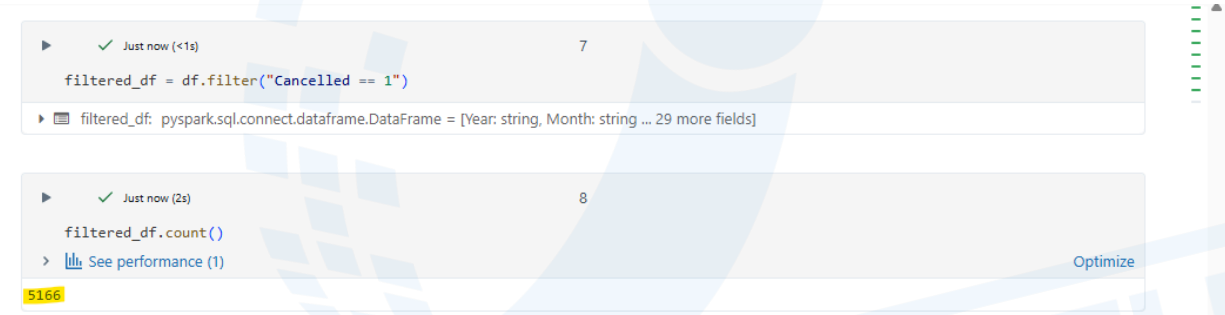

```
display(dbutils.fs.ls("/databricks-datasets/"))
```




```
df = spark.read.option("header",
True).csv("/databricks-datasets/airlines/part-00000")
```



To get a list of cancelled flights:



In Databricks Free Edition :

- DBFS root is disabled → you cannot write to /tmp, /dbfs, /mnt, etc.

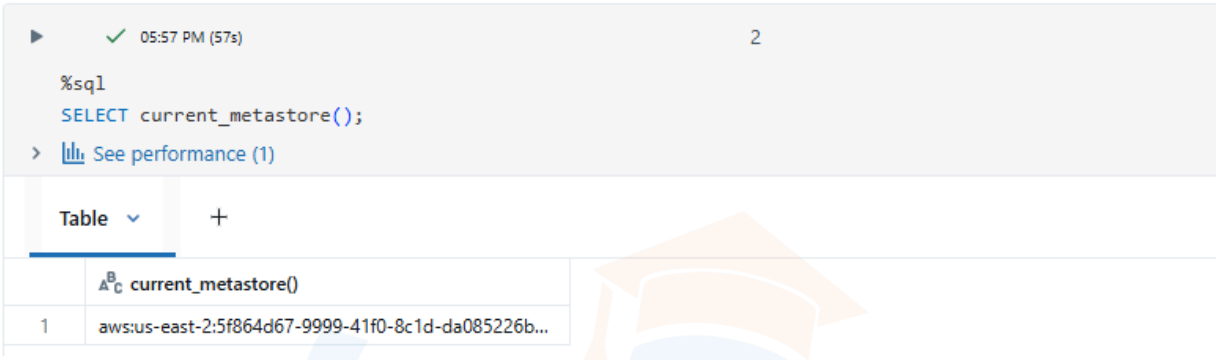
Session 7: Managing Table Data and Metadata in Databricks Using Unity Catalog

In Databricks Free Edition, you do have a metastore, but it's:

- Shared across your workspace only (not Unity Catalog).
- All tables are managed tables (stored in Databricks' backend storage, not in DBFS).
- You cannot change the warehouse directory or inspect the actual storage location.

SQL code

how to check the current metastore

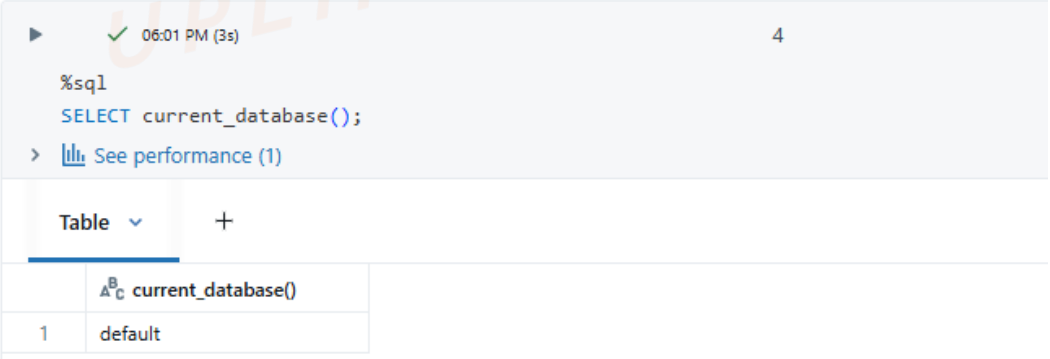


SQL code:

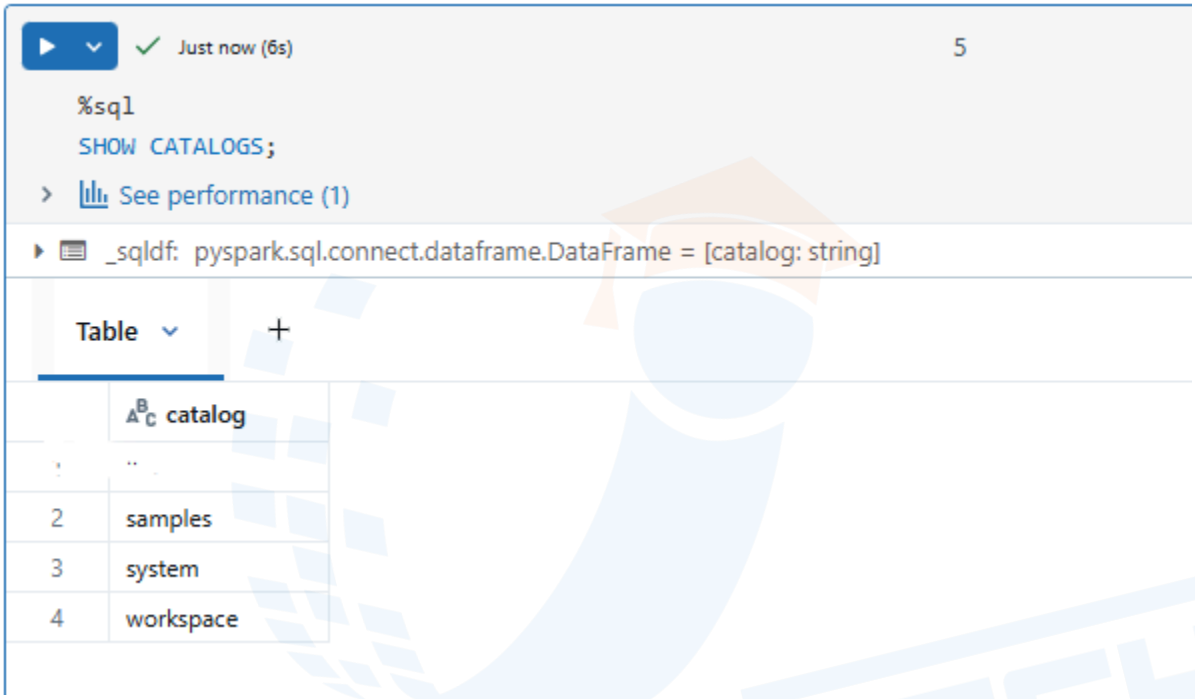
Equivalent code using python:

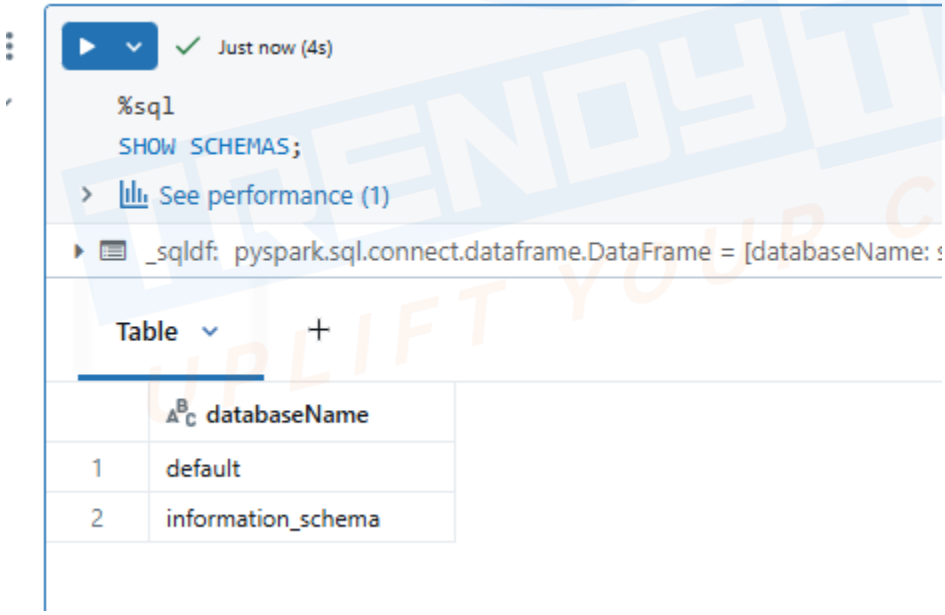
spark.sql("SELECT current_catalog()").show()
spark.sql("SELECT current_schema()").show()

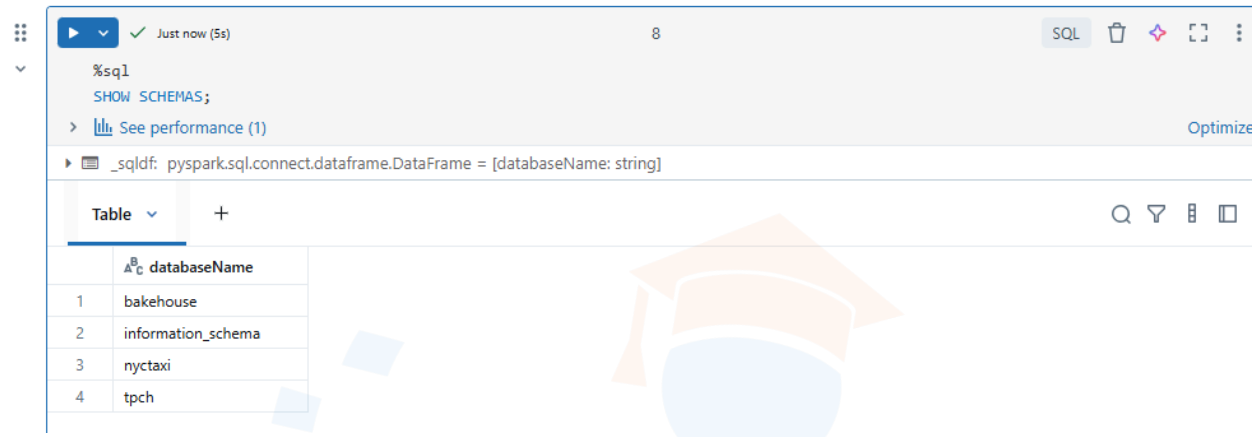
Sql code:



Python equivalent code :
print("Current Database:", spark.catalog.currentDatabase())

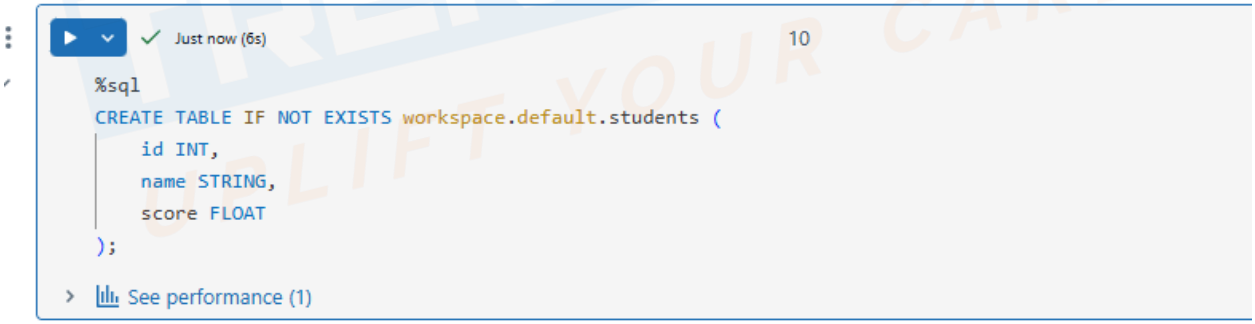






Python equivalent code:
# Switch to catalog `samples`
spark.sql("USE CATALOG workspace")

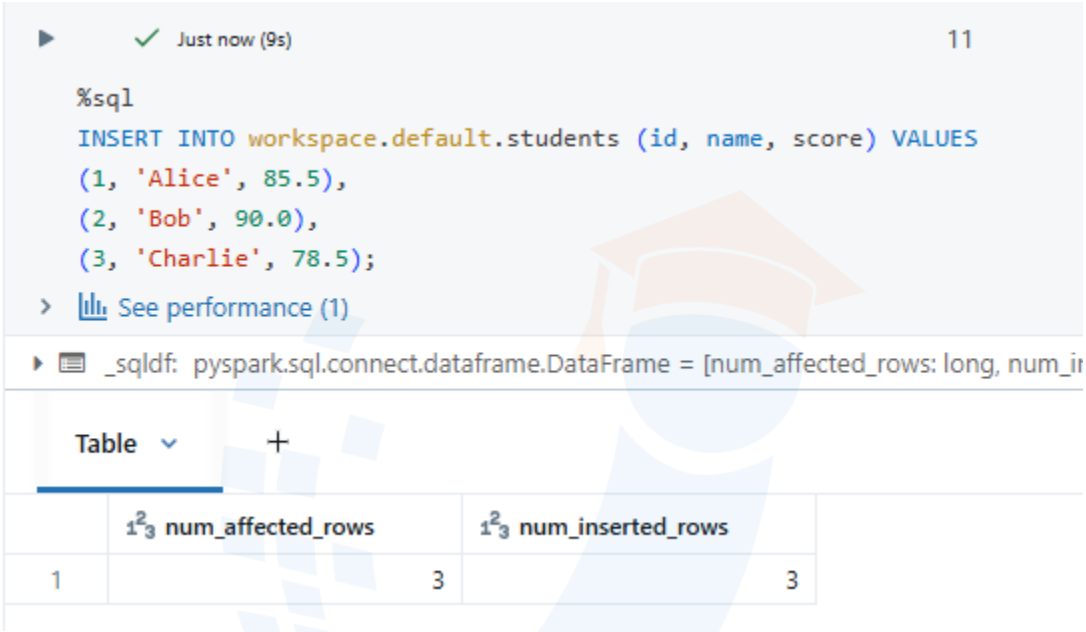
spark.sql("show schemas")
The default catalog in free edition is ‘workspace’



Equivalent python code:
spark.sql("""
CREATE TABLE IF NOT EXISTS workspace.default.students (
  id INT,

```
  name STRING,
  score FLOAT
)
""")
```

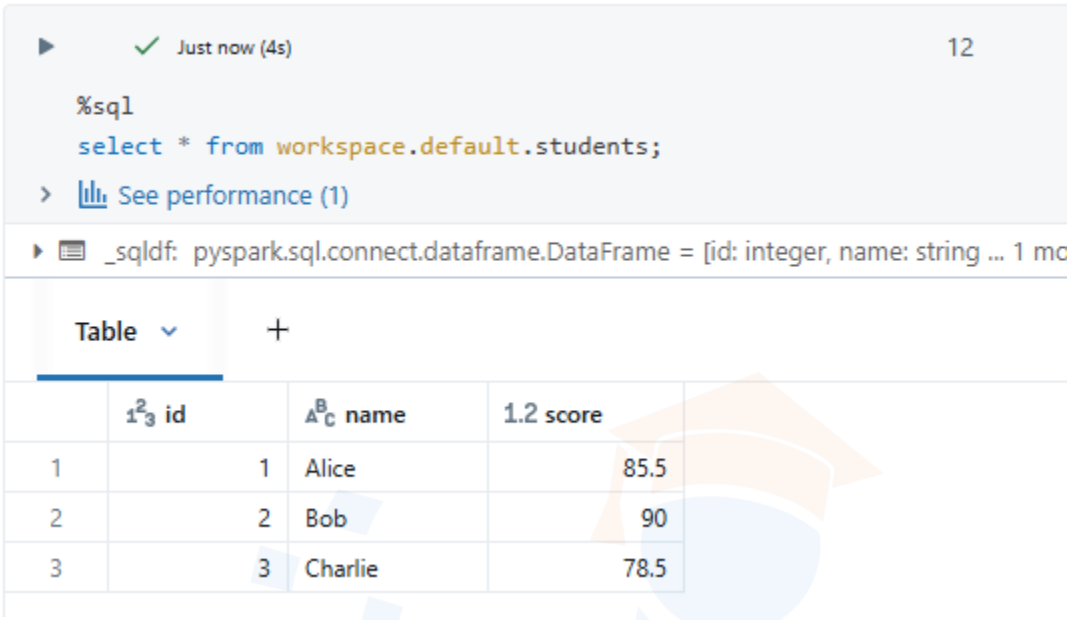
SQL code:



Equivalent Python Code:

```
spark.sql("""
INSERT INTO workspace.default.students (id, name, score) VALUES
(1, 'Alice', 85.5),
(2, 'Bob', 90.0),
(3, 'Charlie', 78.5)
""")
```

SQL code:

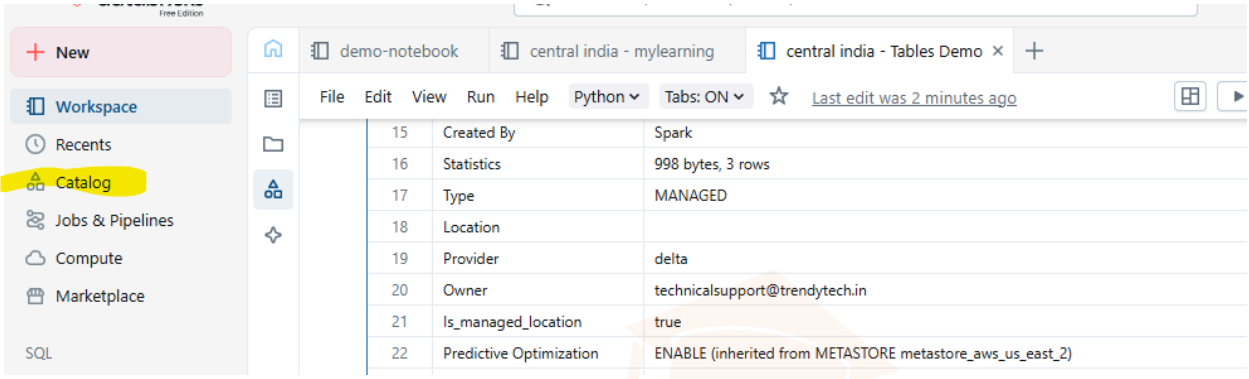


Python equivalent :
df = spark.sql("SELECT * FROM workspace.default.students")
df.show()

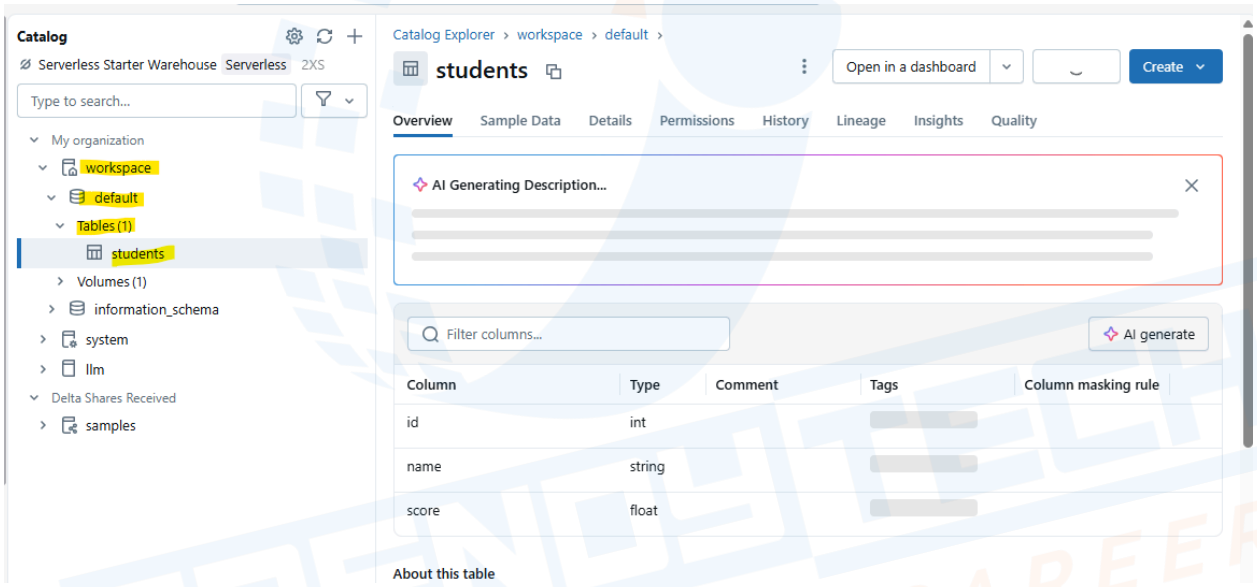
SQL code:

Python equivalent code:
df = spark.sql("DESC EXTENDED workspace.default.students")
df.show(truncate=False)

To see in catalog explorer:
Go to catalog



Catalog → Workspace → default → Tables → students



Hive_metastore is not available in the free edition

How to Use Azure Storage with Databricks Unity Catalog - need azure account again

```
%sql
create catalog my_catalog1
managed location
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog';
```

Python equivalent code:
spark.sql("""
CREATE CATALOG my_catalog1

MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
""")

```
%sql
use catalog my_catalog1;
```

Pyspark:
spark.sql("USE CATALOG my_catalog1")

Creating External Locations in Databricks Using SQL - need azure account again

```
%sql
SHOW EXTERNAL LOCATIONS;
```

Pyspark equivalent code:
df = spark.sql("SHOW EXTERNAL LOCATIONS")
df.show(truncate=False)

```
%sql
SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc;
```

Pyspark:
df = spark.sql("SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc")
df.show(truncate=False)

```
%sql
GRANT CREATE MANAGED STORAGE ON EXTERNAL LOCATION adls_raw_storage_ext_loc TO
`sumit.trendytech02_outlook.com#EXT#@sumittrendytech02outlook.onmicrosoft.com`;

SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc;
```

Pyspark:

Grant permission
spark.sql("""
GRANT CREATE MANAGED STORAGE
ON EXTERNAL LOCATION adls_raw_storage_ext_loc
TO `sumit.trendytech02_outlook.com#EXT#@sumittrendytech02outlook.onmicrosoft.com`
""")

Verify grants

df = spark.sql("SHOW GRANTS ON EXTERNAL LOCATION adls_raw_storage_ext_loc")
df.show(truncate=False)

```
%sql
CREATE CATALOG sales_catalog;
```

Pyspark:
spark.sql("CREATE CATALOG sales_catalog")

```
%sql
CREATE SCHEMA sales_catalog.retaildb;
```

Pyspark:
spark.sql("CREATE SCHEMA sales_catalog.retaildb")

```
%sql
CREATE TABLE sales_catalog.retaildb.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
);
```

Pyspark:
spark.sql("""
CREATE TABLE sales_catalog.retaildb.sales (
  sale_id INT,
  customer_name STRING,
  product STRING,
  quantity INT,
  amount DOUBLE
)
""")

```
%sql
INSERT INTO sales_catalog.retaildb.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
```

```
(3, 'Charlie', 'Phone', 1, 500.00);
```

```
spark.sql("""
INSERT INTO sales_catalog.retaildb.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00)
""")
```

Alternatively, you can also use the dataframe method:

```
from pyspark.sql import Row

# Create sample data
data = [
    Row(1, "Alice", "Laptop", 1, 1200.00),
    Row(2, "Bob", "Tablet", 2, 800.00),
    Row(3, "Charlie", "Phone", 1, 500.00)
]

# Create DataFrame
df = spark.createDataFrame(data, ["sale_id", "customer_name", "product", "quantity", "amount"])

# Append into table
df.write.mode("append").saveAsTable("sales_catalog.retaildb.sales")
```

```
%sql
DESCRIBE EXTENDED sales_catalog.retaildb.sales
```

Pyspark:

```
df = spark.sql("DESCRIBE EXTENDED sales_catalog.retaildb.sales")
df.show(truncate=False)
```

```
%sql
DROP CATALOG IF EXISTS sales_catalog CASCADE;
```

Pyspark:

```
spark.sql("DROP CATALOG IF EXISTS sales_catalog CASCADE")
```

```
%sql
CREATE EXTERNAL LOCATION adls_raw_storage_catalog_ext_loc URL
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
WITH (CREDENTIAL adls_raw_storage_cred)
```

```
COMMENT 'external location catalog level for raw sales data in adls';
```

Pyspark:

```
spark.sql("""
CREATE EXTERNAL LOCATION adls_raw_storage_catalog_ext_loc
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location catalog level for raw sales data in adls'
""")
```

```
%sql
CREATE CATALOG sales_catalog
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog';
```

Pyspark:

```
spark.sql("""
CREATE CATALOG IF NOT EXISTS sales_catalog
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/catalog'
""")
```

```
%sql
CREATE SCHEMA sales_catalog.retaildb;
```

Pyspark:

```
spark.sql("CREATE SCHEMA IF NOT EXISTS sales_catalog.retaildb")
```

```
%sql
CREATE TABLE sales_catalog.retaildb.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
);
```

Pyspark:

```
spark.sql("""
CREATE TABLE IF NOT EXISTS sales_catalog.retaildb.sales (
   sale_id INT,
```

```
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
)
""")
```

```
%sql
INSERT INTO sales_catalog.retaildb.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00);
```

Pyspark;

```
spark.sql("""
INSERT INTO sales_catalog.retaildb.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00)
""")
```

Alternatively, using a DataFrame in PySpark:

```
from pyspark.sql import Row

data = [
    Row(1, "Alice", "Laptop", 1, 1200.00),
    Row(2, "Bob", "Tablet", 2, 800.00),
    Row(3, "Charlie", "Phone", 1, 500.00)
]

df = spark.createDataFrame(data, ["sale_id", "customer_name", "product", "quantity", "amount"])

df.write.mode("append").saveAsTable("sales_catalog.retaildb.sales")
```

```
%sql
DESCRIBE EXTENDED sales_catalog.retaildb.sales
```

Pyspark:

```
df = spark.sql("DESCRIBE EXTENDED sales_catalog.retaildb.sales")
df.show(truncate=False)
```

```
%sql
CREATE EXTERNAL LOCATION adls_raw_storage_schema_ext_loc URL
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location schema level for raw sales data in adls'
```

Pyspark:
spark.sql("""
CREATE EXTERNAL LOCATION adls_raw_storage_schema_ext_loc
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location schema level for raw sales data in adls'
""")

```
%sql
CREATE SCHEMA sales_catalog.retaildb1
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema';
```

Pyspark:
spark.sql("""
CREATE SCHEMA IF NOT EXISTS sales_catalog.retaildb1
MANAGED LOCATION
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/schema'
""")

```
%sql
CREATE TABLE sales_catalog.retaildb1.sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
);
```

Pyspark:
spark.sql("""
CREATE TABLE IF NOT EXISTS sales_catalog.retaildb1.sales (
  sale_id INT,
  customer_name STRING,
  product STRING,

```
    quantity INT,
    amount DOUBLE
)
""")
```

```
%sql
INSERT INTO sales_catalog.retaildb1.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00);
```

Pyspark:

```
spark.sql("""
INSERT INTO sales_catalog.retaildb1.sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00)
""")
```

Alternate dataframe method:

```
from pyspark.sql import Row

data = [
    Row(1, "Alice", "Laptop", 1, 1200.00),
    Row(2, "Bob", "Tablet", 2, 800.00),
    Row(3, "Charlie", "Phone", 1, 500.00)
]

df = spark.createDataFrame(data, ["sale_id", "customer_name", "product", "quantity", "amount"])

df.write.mode("append").saveAsTable("sales_catalog.retaildb1.sales")
```

Managed vs External Tables in Databricks - need azure account

```
%sql
DROP TABLE sales_catalog.retaildb.sales;
```

Pyspark;

```
spark.sql("DROP TABLE IF EXISTS sales_catalog.retaildb.sales")
```

```
%sql
SHOW TABLES DROPPED IN sales_catalog.retaildb;
```

Pyspark:
df = spark.sql("SHOW TABLES DROPPED IN sales_catalog.retaildb")
df.show(truncate=False)

```
%sql
select * from sales_catalog.retaildb.sales;
```

Pyspark:
df = spark.sql("SELECT * FROM sales_catalog.retaildb.sales")
df.show(truncate=False)

```
%sql
UNDROP TABLE sales_catalog.retaildb.sales;
```

Pyspark:
spark.sql("UNDROP TABLE sales_catalog.retaildb.sales")

```
%sql
CREATE EXTERNAL LOCATION adls_storage_external_data_loc URL
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location for handling external data in adls';
```

Pyspark:
spark.sql("""
CREATE EXTERNAL LOCATION adls_storage_external_data_loc
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location for handling external data in adls'
""")

```
%sql
CREATE TABLE sales (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
)
location
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external';
```

Pyspark:

```
spark.sql("""
CREATE TABLE sales (
   sale_id INT,
   customer_name STRING,
   product STRING,
   quantity INT,
   amount DOUBLE
)
LOCATION 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external'
""")
```

```
%sql
desc extended sales;
```

Pyspark:

```
df = spark.sql("DESC EXTENDED sales")
df.show(truncate=False)
```

```
%sql
INSERT INTO sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00);
```

Pyspark:

```
spark.sql("""
INSERT INTO sales VALUES
(1, 'Alice', 'Laptop', 1, 1200.00),
(2, 'Bob', 'Tablet', 2, 800.00),
(3, 'Charlie', 'Phone', 1, 500.00)
""")
```

```
%sql
select * from sales;
```

Pyspark:

```
df = spark.sql("SELECT * FROM sales")
df.show(truncate=False)
```

```
%sql
DROP TABLE IF EXISTS sales;
```

Pyspark:

```
spark.sql("DROP TABLE IF EXISTS sales")
```

```
%sql
CREATE EXTERNAL LOCATION adls_storage_external_sales_data_loc URL
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales
'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location for handling external sales data in adls'
```

Pyspark:

```
spark.sql("""
CREATE EXTERNAL LOCATION adls_storage_external_sales_data_loc
URL 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales'
WITH (CREDENTIAL adls_raw_storage_cred)
COMMENT 'external location for handling external sales data in adls'
""")
```

```
%sql
CREATE TABLE sales_ext (
    sale_id INT,
    customer_name STRING,
    product STRING,
    quantity INT,
    amount DOUBLE
)
USING CSV
location
'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales
';
```

Pyspark:

```
spark.sql("""
CREATE TABLE IF NOT EXISTS sales_ext (
   sale_id INT,
   customer_name STRING,
   product STRING,
   quantity INT,
   amount DOUBLE
)
USING CSV
LOCATION 'abfss://unitycatalog@databricksdemo101sa.dfs.core.windows.net/external_1/sales'
""")
```

```
%sql
select * from sales_ext;
```

Pyspark:
df = spark.sql("SELECT * FROM sales_ext")
df.show(truncate=False)

```
%sql
REFRESH TABLE training_centralus_dbws.default.sales_ext;
```

Pyspark:
spark.catalog.refreshTable("training_centralus_dbws.default.sales_ext")